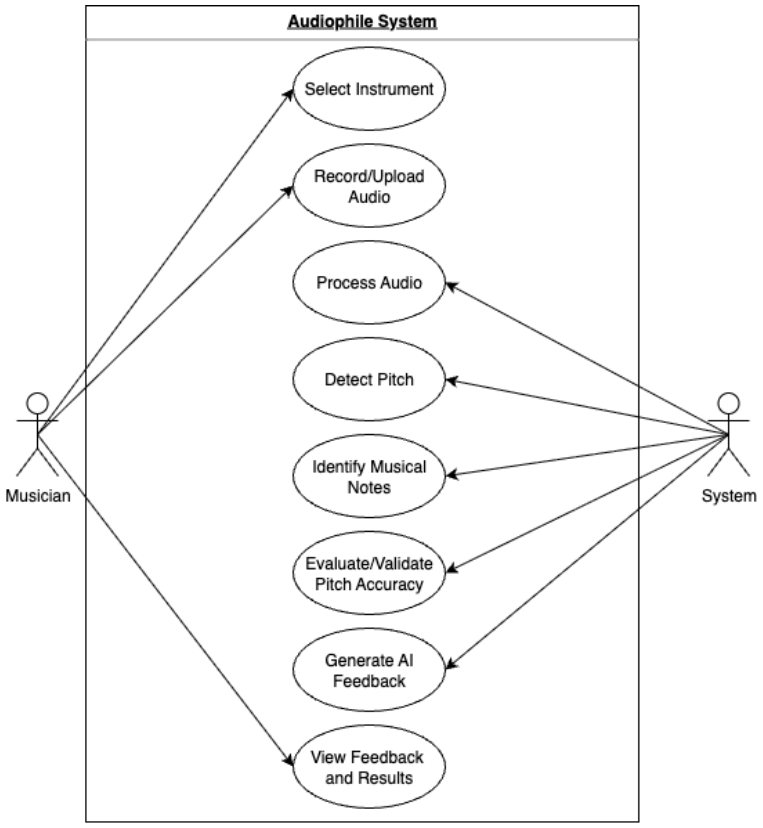
Audiophile – System Requirements Document

1. Overview of Audiophile's Functional Requirements

Audiophile is a web-based system designed to assist beginner and intermediate musicians during independent instrument practice. The system analyzes instrument audio and provides immediate feedback regarding pitch accuracy.

Musicians can record audio directly through the web interface or upload an existing audio file. The system processes the audio, detects the pitch frequency, maps it to the closest letter notation, and determines whether the note is in tune, sharp, or flat. The system also validates detected pitches based on the selected instrument's valid pitch range to improve reliability. After evaluating pitch accuracy, Audiophile generates personalized feedback using an AI-based feedback generation component integrated through the Groq API. The system provides clear, non-repetitive explanations that help musicians understand how to adjust their playing.

1.1. Use Case Diagram



Musicians can select their own instrument, record live audio input, submit an audio file, and view their feedback results. The system will process the audio, detect individual pitches and match them with letter notations, evaluate pitch accuracy, and generate personalized AI feedback for the musician.

2. Audiophile User Stories

2.1. Select Instrument

As a musician, I want to select the instrument I am practicing so that the system can analyze my performance based on the correct instrument settings.

2.1.1. Pre-Conditions

The user has accessed the web application and can view the instrument selection display. The list of supported instruments is loaded, and the system is ready to accept the user's input.

2.1.2. Post-Conditions

The selected instrument is stored in the session, and the system becomes configured with that instrument in mind.

2.1.3. Epic Structure

This story is simple and focused. It only involves choosing an instrument from a list. It does not require further breakdown because it represents one clear user action.

2.2. Record Audio

As a musician, I want to record or upload my audio performance so that I can receive feedback without needing a teacher present.

2.2.1. Pre-Conditions

An instrument has been selected and the user has accepted microphone permissions. The recording interface is active and ready to receive audio.

2.2.2. Post-Conditions

Audio is recorded with a temporary audio file in the system. The user sees that the recording has concluded, and the file is ready to be processed.

2.2.3. Epic Structure

This story can be considered an epic because recording audio involves multiple steps and components. It will be broken down into smaller user stories:

- Start recording audio
- Stop recording audio
- Save recorded audio

2.3. Process Audio

As the system, I will effectively process recorded/uploaded audio to accurately detect pitch and remove noise.

2.3.1. <u>Pre-Conditions</u>

A valid MP3 audio recording file exists. The selected instrument settings are also loaded and the audio processing module can be used.

2.3.2. <u>Post-Conditions</u>

The audio is cleaned and normalized. The audio is converted into a format ready for pitch detection. The processed audio is stored in the system.

2.3.3. <u>Epic Structure</u>

This story involves technical complexity in processing audio and needs to be broken down into smaller, more technical stories:

- Reducing background noise
- Converting audio format
- Normalizing sound levels

2.4. Detect Pitch

As the system, I will ensure pitch is accurate so that the exact frequency and letter notation can be determined.

2.4.1. <u>Pre-Conditions</u>

Processed audio is available and the correct instrument settings are loaded.

2.4.2. <u>Post-Conditions</u>

The fundamental frequency in hertz (Hz) is found. Detected pitch values are stored.

2.4.3. <u>Epic Structure</u>

Although technically complex, this story focuses on one specific outcome: detecting pitch. It does not need to be broken down at the requirement level.

2.5. Identify Letter Notation

As the system, I will identify the letter notation so that the musician can confirm the notes they are playing are the correct ones.

2.5.1. <u>Pre-Conditions</u>

Pitch frequency was successfully detected. The pitch-to-note mapping is available, with a reference table available as well.

2.5.2. <u>Post-Conditions</u>

The detected frequency is mapped to the closest note/letter and stored. The system saves the note information for evaluation.

2.5.3. <u>Epic Structure</u>

This story is focused on converting pitch into letter notation. It is a clear and single function, so it does not require decomposition.

2.6. Evaluate Pitch Accuracy

As the system, I will evaluate pitch accuracy/intonation and give feedback on whether the note was played in tune, sharp, or flat.

2.6.1. <u>Pre-Conditions</u>

The note has been identified. The detected and expected pitch frequencies are available. Related information and rules regarding the selected instrument are also loaded.

2.6.2. <u>Post-Conditions</u>

The pitch is evaluated as in tune, sharp, or flat. The deviation from the proper pitch is calculated. The result is stored and out-of-tune pitches are flagged.

2.6.3. <u>Epic Structure</u>

This story involves checking whether the note is correct, sharp, or flat. It is focused and manageable as one unit and does not need to be broken down.

2.7. Validate Performance

As the system, I want to validate the audio recording so the user can confirm they are following the on-screen instructions/guidance/exercise.

2.7.1. *<u>Pre-Conditions</u>*

Pitch accuracy has been evaluated and the selected instrument is set.

2.7.2. *<u>Post-Conditions</u>*

The detected pitch is validated and the system determines if the pitch is within an acceptable deviation. Wrong notes are flagged, and the results are stored.

2.7.3. *<u>Epic Structure</u>*

This story checks whether the performed note matches the expected note. It represents one clear validation action and does not require decomposition.

2.8. *Generate Feedback*

As the system, I want to generate personalized feedback on the user's audio recording so they can improve their tone, intonation, technique, and knowledge.

2.8.1. *<u>Pre-Conditions</u>*

Performance validation and pitch evaluation results are available in the system. The Groq API connection and AI feedback modules are active.

2.8.2. *<u>Post-Conditions</u>*

Feedback text is generated and stored. Any errors from the API are logged and handled.

2.8.3. *<u>Epic Structure</u>*

This story is considered an epic because of its technical demands and multi-step process. It will require further breakdown into smaller stories:

- Create feedback message
- Send data to AI
- Display generated feedback

2.9. *View Feedback Results*

As a musician, I want clear access to personalized feedback so that I can effectively improve my skillset, knowledge, and performance.

2.9.1. *<u>Pre-Conditions</u>*

Feedback has been generated successfully. Evaluation results are available as well.

2.9.2. *<u>Post-Conditions</u>*

Feedback is displayed on the web application with results for pitch and note accuracy visible. The user can review the results and the session remains active to be reused if needed.

2.9.3. *<u>Epic Structure</u>*

This story involves displaying results on the screen. It is simple and focused, so it does not need to be broken down.

3. Non-functional Requirements

3.1. Product requirements

3.1.1. The Audiophile system shall have at least 99% monthly uptime, excluding maintenance.

3.1.2. The system shall process audio and display AI feedback within 1 minute after submission of an audio recording that is between 30 seconds and 2 minutes long.

3.1.3. All audio uploads/recordings shall be encrypted in transit (HTTPS) using industry-standard encryption.

3.1.4. Feedback messages shall follow defined pedagogical patterns in natural language, including at least one strength and one 'improvement' suggestion per audio submission.

3.1.5. The web application UI shall allow a user to submit/record audio and receive feedback within three clicks from the main screen.

3.1.6. The system shall remain accessible within the WCAG 2.1 Level AA standard.

3.2. Organizational requirements

3.2.1. Users of the Audiophile system shall authenticate themselves using a valid email address.

3.2.2. All user interactions, submissions, and AI feedback messages shall be logged with timestamps for audit and debugging purposes.

3.3. External requirements

3.3.1. The system shall support current versions of major browsers on Windows and MacOS without requiring extra plugins or action from the user.

3.3.2. The system shall comply with FERPA regulations and provide mechanisms for account deletion and data export.

3.3.3. The system shall clearly disclose when feedback is generated by AI, and how recordings may be used for model development, in line with relevant AI transparency guidelines.

4. Glossary

4.1. Musical Terms:

- **Pitch** - The perceived frequency of a sound.
- **Frequency (Hertz/Hz)** - The number of sound wave cycles per second.
- **Standard Reference** - A predefined pitch used for tuning by comparison. Standard tuning for guitar is A4 = 440 Hertz/Hz.
- **Sharp** - A pitch higher than the standard reference.
- **Flat** - A pitch lower than the standard reference.
- **In Tune** - A pitch within an acceptable deviation range of the standard reference.
- **Instrument Range** - The valid pitch range playable by a specific instrument.
- **Letter Notation** - A specific pitch associated with a specific frequency represented with letters (A-G) and modified by sharps (#) or flats (♭).

4.2. Audio Processing Terms:

- **Pitch Detection** - The process of extracting frequency from an audio signal.
- **Preprocessing** - Preparing audio data before analysis (normalizing, formatting).

4.3. System & Technical Terms:

- **Groq API** - External AI service used to generate personalized feedback.
- **Backend API** - The communication layer between frontend and processing modules.

- **User Interface (UI)** - The visual interface at which musicians interact with the system.
- **Hugging Face** - An open-source platform that provides machine learning models, tools for APIs for tasks such as audio processing, natural language processing, and AI deployment.
- **Python** - A high-level programming language used in the Audiophile system for audio processing, pitch detection algorithms, and backend logic.
- **React** - A JavaScript library used to build the system's frontend user interface through reusable components.
- **JavaScript** - A programming language that is used to implement frontend logic and client side functionality for the system.
- **Node.js** - A JavaScript runtime environment used to execute server-side code.